Appendix C: IEEE 830 Template

Software Requirements Specification

for

Simple Digital Thermometer

Version «#5»

Prepared by Adam Omieciński

Lodz University of Technology

«10/06/2026»

Table of Contents

Revision History

Name	Date	Reason for Changes	Version
«Adam Omieiciński»	«27/04/2026»	«Initial version»	«#1»
«Adam Omieiciński»	«08/05/2026»	«UML implemented»	«#2»
«Adam Omieiciński»	«15/05/2026»	«Added System Architecture diagram»	«#3»
«Adam Omieiciński»	«22/05/2026»	«Added Workflow diagram»	«#4»
«Adam Omieiciński»	«10/06/2026»	«Final Version»	«#5»

1. Introduction

1.1. Purpose

This document identifies the software requirements and design for the Simple Digital Thermometer product. It covers the scope of the basic temperature monitoring system intended for home use.

1.2. Document Conventions

This document follows standard IEEE 830 conventions. The word "shall" indicates a mandatory requirement, while "should" indicates a desirable but not strictly mandatory feature. All functional requirements are prioritized equally unless explicitly stated otherwise.

1.3. Intended Audience and Reading Suggestions

This document is intended for software developers, hardware engineers, system testers, and project evaluators. Readers are advised to start with Section 2 for a general overview of the system's capabilities before proceeding to Sections 4, 5, and 6 for specific technical requirements and constraints.

1.4. Product Scope

The software will operate a simple digital thermometer. Its purpose is to read temperature data from a sensor and display it clearly to the user. The goal is to provide a reliable, easy-to-read, and fast temperature measurement tool.

1.5. References

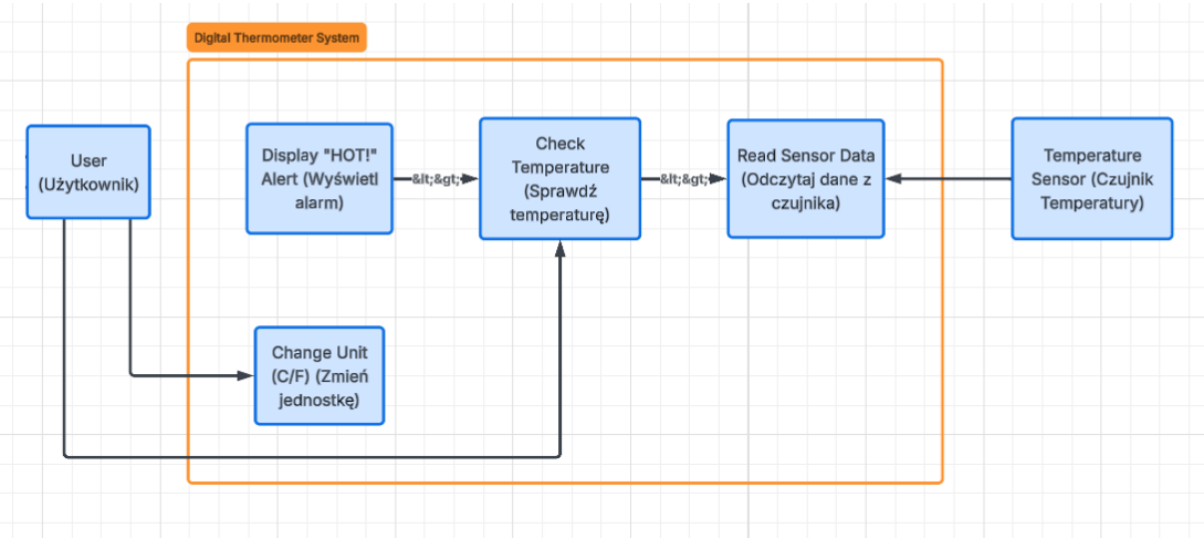
IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

Software Engineering Lab Manuals, Lodz University of Technology.

2. Overall Description

2.1. Product Perspective

The Simple Digital Thermometer is a new, self-contained system. It is an independent application running on a basic microcontroller or local environment. It does not replace any existing systems and operates completely offline, interfacing directly with a physical temperature sensor and a display module.



2.2. Product Features

Read temperature from a sensor.

Display temperature on a screen.

Alert the user if the temperature is too high.

2.3. User Classes and Characteristics

The main user class is a standard home user with no technical expertise. The system must be fully automated and require zero configuration.

2.4. Operating Environment

The software will operate on a basic microcontroller or local PC environment, interfacing with a standard digital temperature sensor.

2.5. Design and Implementation Constraints

The system is constrained to operate as a local desktop application. It must execute its logic and graphical user interface (GUI) without requiring any external network connections or cloud-based databases.

2.6. User Documentation

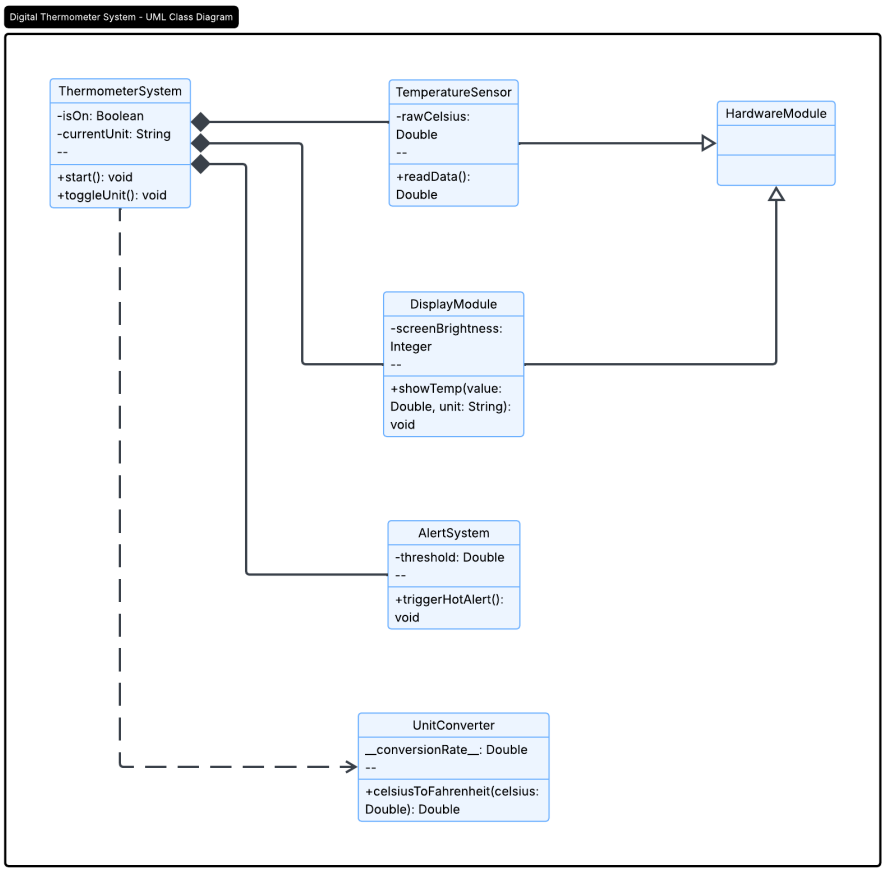
A standard digital README file or an in-app "Help" menu will be provided. It will include instructions on how to launch the application executable and how to use the interface buttons to toggle between temperature units.

2.7. Assumptions and Dependencies

It is assumed that the host computer runs a standard operating system (e.g., Windows, macOS, or Linux) and that the user has the basic runtime environment installed (if required by the chosen programming language). It is also assumed that the application will receive temperature data either from a simulated data stream or a standard plug-and-play USB sensor.

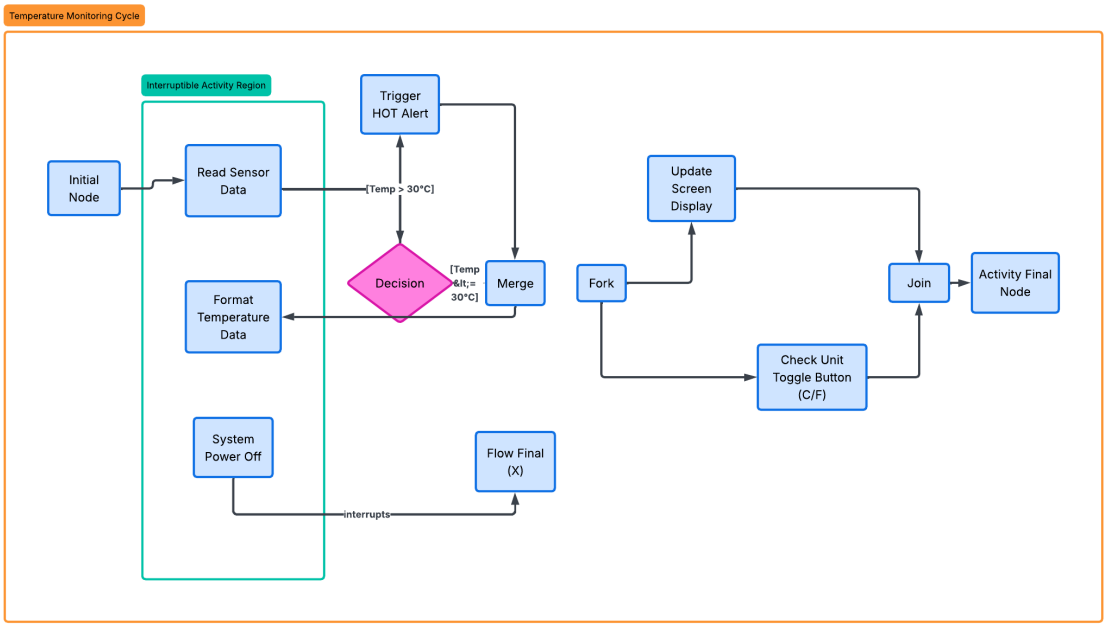
2.8. System Architecture (Class Diagram)

This diagram outlines the static software architecture for the Simple Digital Thermometer. It shows the main orchestrator class (ThermometerSystem) managing hardware abstraction layers, specifically the TemperatureSensor and DisplayModule. It also includes utility components, such as a UnitConverter for temperature scale adjustments and an AlertSystem to handle high-temperature warnings.



2.9. System Workflow (Activity Diagram)

This activity diagram models the continuous operational workflow of the Simple Digital Thermometer. It maps the main execution path from reading sensor data to updating the display. It highlights the decision logic for triggering the high-temperature alert and utilizes concurrent processing (Fork/Join) to handle screen updates while simultaneously listening for user input (unit toggle). An interruptible region is included to handle sudden power loss.



3. System Features

3.1. Temperature Measurement

The system constantly monitors the environment and extracts temperature data to keep the user informed.

4. External Interface Requirements

4.1. User Interfaces Overview

The user interface consists of a simple digital display (either a physical LCD or a basic GUI window on a computer screen) that shows the current temperature as a number.

4.2. Hardware Interfaces

The software relies on standard personal computer hardware. It requires basic input/output devices (a monitor for the graphical interface, and a mouse/keyboard for interaction). If a physical sensor is used, it will interface via a standard USB port; otherwise, no specialized hardware is required.

4.3. Software Interfaces

The application will utilize standard OS-level libraries to render the graphical user interface (GUI) and process user inputs. It will run independently, requiring only the standard system APIs and potentially a common runtime environment (e.g., Python, Java JRE, or .NET framework) depending on the final implementation language.

5. System Features/Modules

5.1. Temperature Monitoring System

5.1.1 Description and Priority

This module is responsible for fetching, processing, and displaying the temperature. Priority: High.

5.1.2 Stimulus/Response Sequences

Condition: The user launches the application executable.

Response: The application window opens, initializes the data stream, and begins continuously displaying the current temperature on the screen.

Stimulus: The user clicks the "Switch Unit (C/F)" button on the graphical interface.

Response: The system immediately converts the current temperature value and updates the displayed text from Celsius to Fahrenheit (or vice versa).

5.1.3 Functional Requirements

REQ-1.1: The system shall read the current temperature data from the sensor..

REQ-1.2: The system shall display the measured temperature in degrees Celsius.

REQ-1.3: The system shall update the displayed temperature every 2 seconds.

REQ-1.4: The system shall display a warning message "HOT!" when the temperature exceeds 30°C.

REQ-1.5: The system shall allow the user to switch the display unit from Celsius to Fahrenheit using a designated button or key press.

6. Nonfunctional Requirements

6.1. Performance

NF-1.1: System shall respond to the user's unit-switch command in < 1 second.

NF-1.2: The system shall successfully boot up and display the first reading in < 5 seconds.

NF-1.3: The system shall maintain a temperature measurement accuracy of +/- 1°C.

6.2. Reliability & Environment

NF-2.1: The system shall maintain 99% uptime during continuous operation.

NF-2.2: The system shall operate stably on a standard 5V power supply.